



UCSD ECE 268 - Security of Hardware Embedded Systems

GGM Tree Construction: Spongnet and Keccak

Group 2

Anjana Manoj, Mark Sui, Nikhita Neelakanta
June 11th, 2026

Instructor:

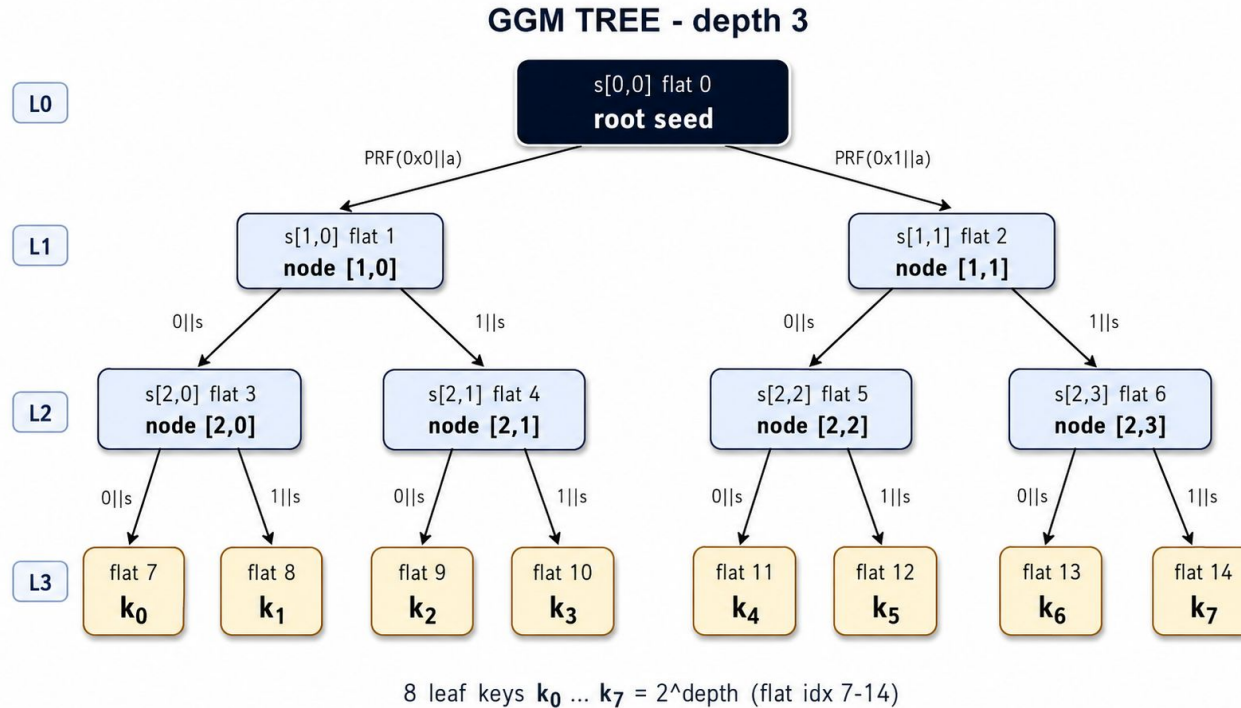
Prof. Farinaz Koushanfar

farinaz@ucsd.edu

UC San Diego

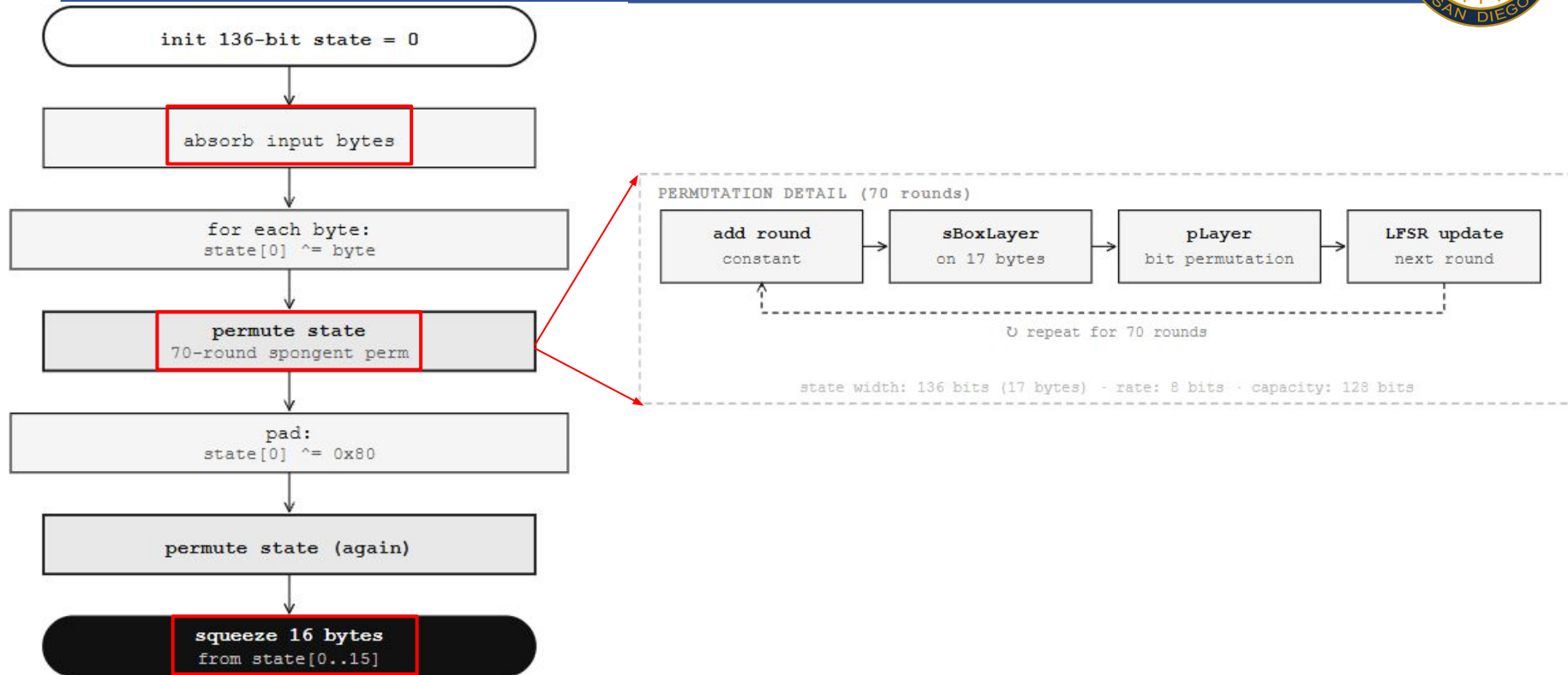
JACOBS SCHOOL OF ENGINEERING
Electrical and Computer Engineering

What Is a GGM Tree?



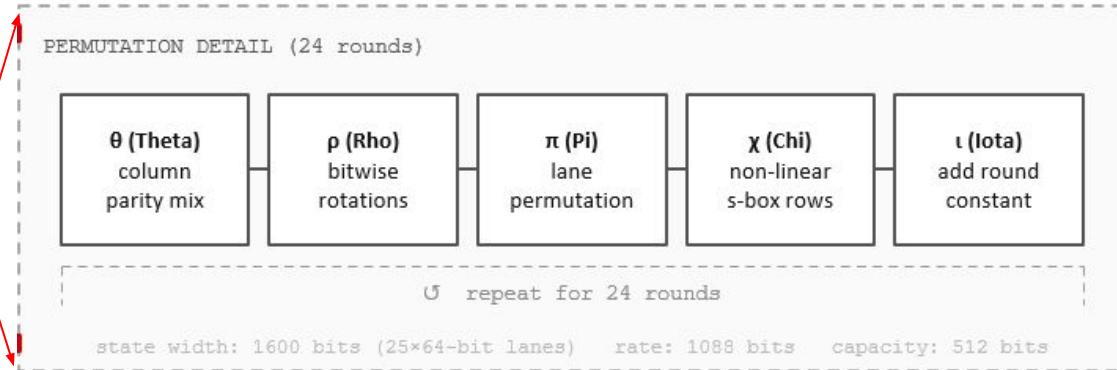
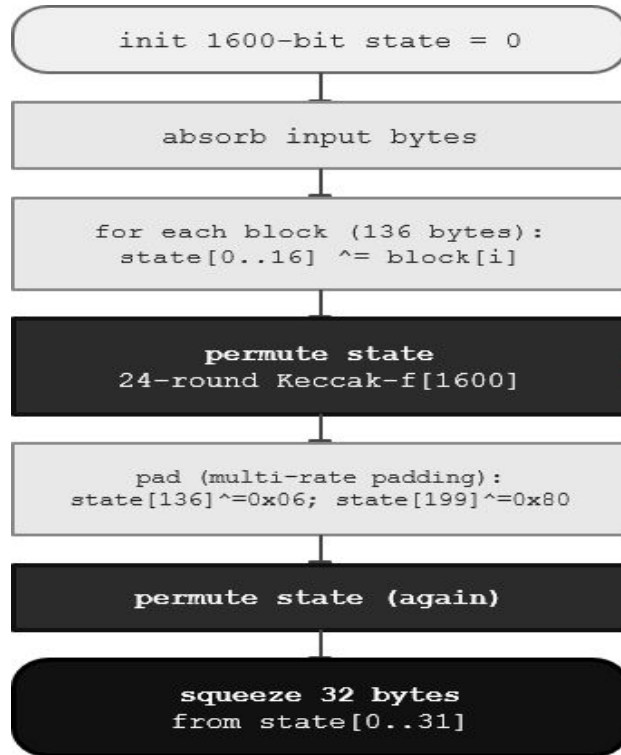


Spongint-128





Keccak-f1600





PRF GPU Implementation Optimisations

Spongant-128

1 S-box

Byte-level constant memory lookup
Eliminates warp divergence

2 pLayer

Doubly-unrolled b and k loops
Compile-time indices, tmp[] in regs

3 Single sync

One sync after all levels
Stream ordering ensures correctness

Keccak-f1600

1 Rho+Pi

25 literal assignments, no loop
Compile-time indices, tmp[] in regs
Zero DRAM traffic

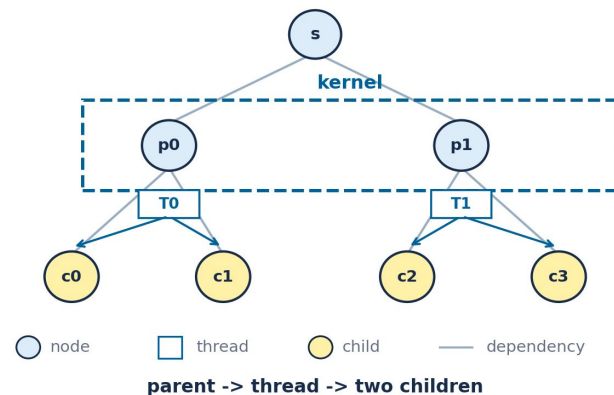
2 Outer loop

#pragma unroll 1 on 24-round loop
One body in l-cache, not 24 copies



Shared GPU parallelisation strategy

- **Flat BFS memory layout:**
 - Node (l, i) stored at flat index $2^l - 1 + i$. Threads at the same level read adjacent parent nodes $\rightarrow$ coalesced global memory access.
- **One thread per parent node:**
 - Each thread reads one seed, writes two children. No cross-thread communication needed.
- **Level-by-level kernel launch, not one thread per leaf:**
 - At depth 12, a path-traversal approach would redo all 12 levels per leaf = $12 \times$ redundant work for the same result.
- **Single cudaDeviceSynchronize() after all levels:**
 - kernels for consecutive levels are submitted back-to-back; CUDA stream ordering guarantees level $l+1$ cannot start until level l finishes.





Experimental Setup

Platform

- NVIDIA GTX 1080 Ti
- 3584 CUDA cores
- CUDA C++ implementation

Tree Depths

- Depth 8 → 256 leaves
- Depth 12 → 4,096 leaves
- Depth 16 → 65,536 leaves
- Depth 20 → 1,048,576 leaves

Metrics

- Runtime in milliseconds
- Throughput in leaves per second
- GPU speedup over CPU baseline

PRF Backends

- Spongnet-128 with 16-byte seeds
- Keccak-f1600 / SHA3-256 with 32-byte seeds
- Both use the same GGM tree API

Speedup = GPU leaves/sec ÷ CPU leaves/sec.
All results use full-tree construction.

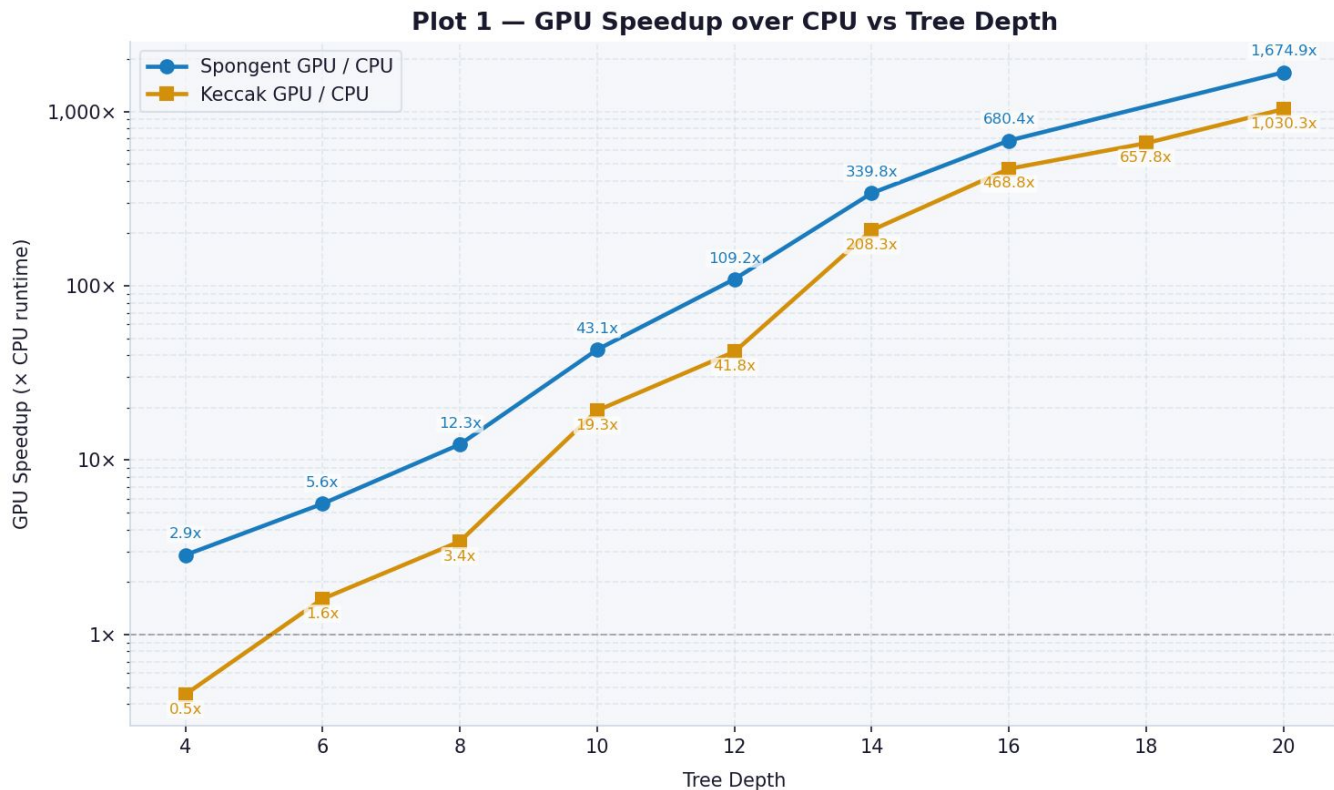


Depth Analysis - GPU vs CPU

DEPTH	LEAVES	CPU TIME (MS)	GPU TIME (MS)	GPU THROUGHPUT (L/S)	INDIVIDUAL GPU/CPU SPEEDUP
KECCAK ARCHITECTURE (NVIDIA GEFORCE GTX 1080 TI)					
8	256	1.311	0.383	669,161.46	3.43x
12	4,096	20.581	0.492	8,326,161.79	41.84x
16	65,536	320.665	0.684	95,835,578.72	468.92x
20	1,048,576	5,346.094	5.189	202,068,562.50	1,030.23x
SPONGENT ARCHITECTURE (NVIDIA GEFORCE GTX 1080 TI)					
8	256	727.317	59.028	4,336.90	12.32x
12	4,096	11,859.152	108.581	37,723.06	109.22x
16	65,536	179,874.245	264.376	247,889.16	680.38x
20	1,048,576	2,804,658.541	1,674.492	6,262,05.28	1,674.93x

*Data collected on NVIDIA GeForce GTX 1080 Ti vs Intel i7-12700K

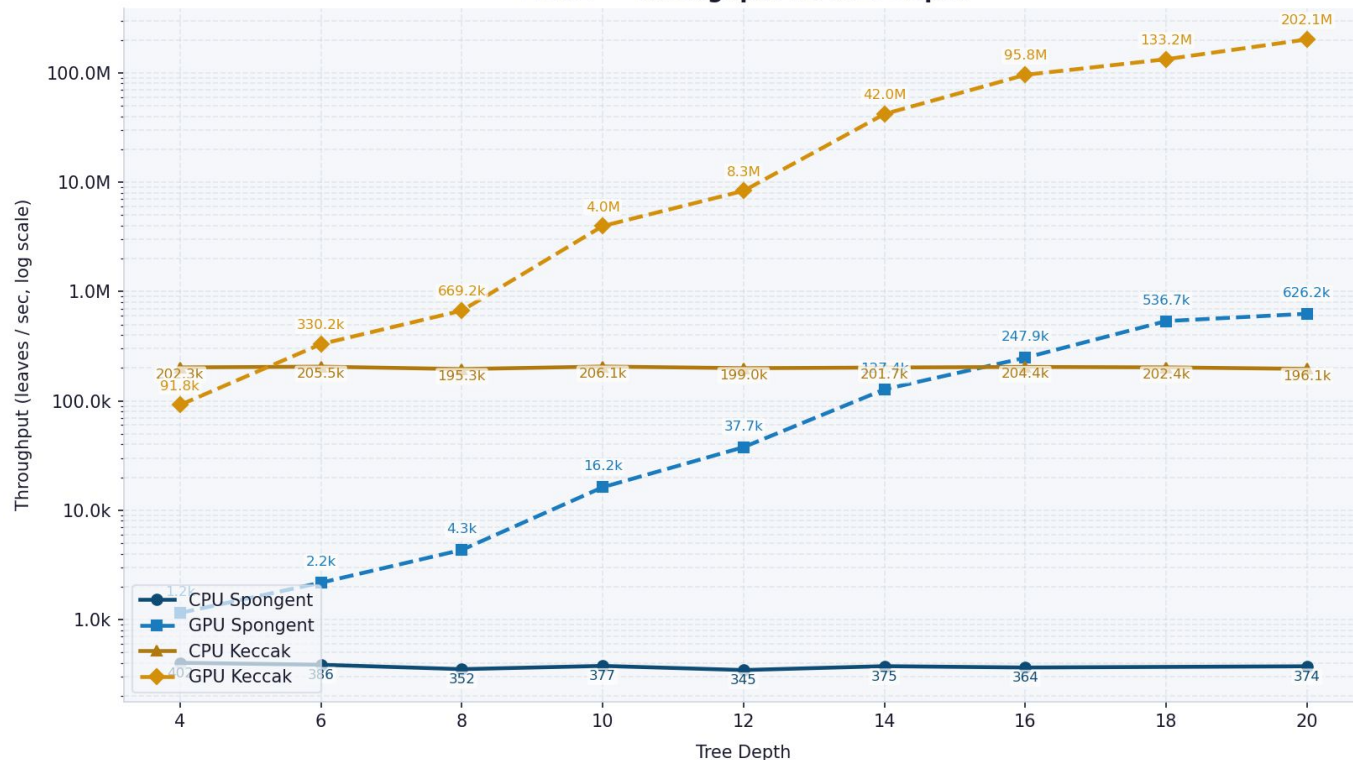
Benchmark Results



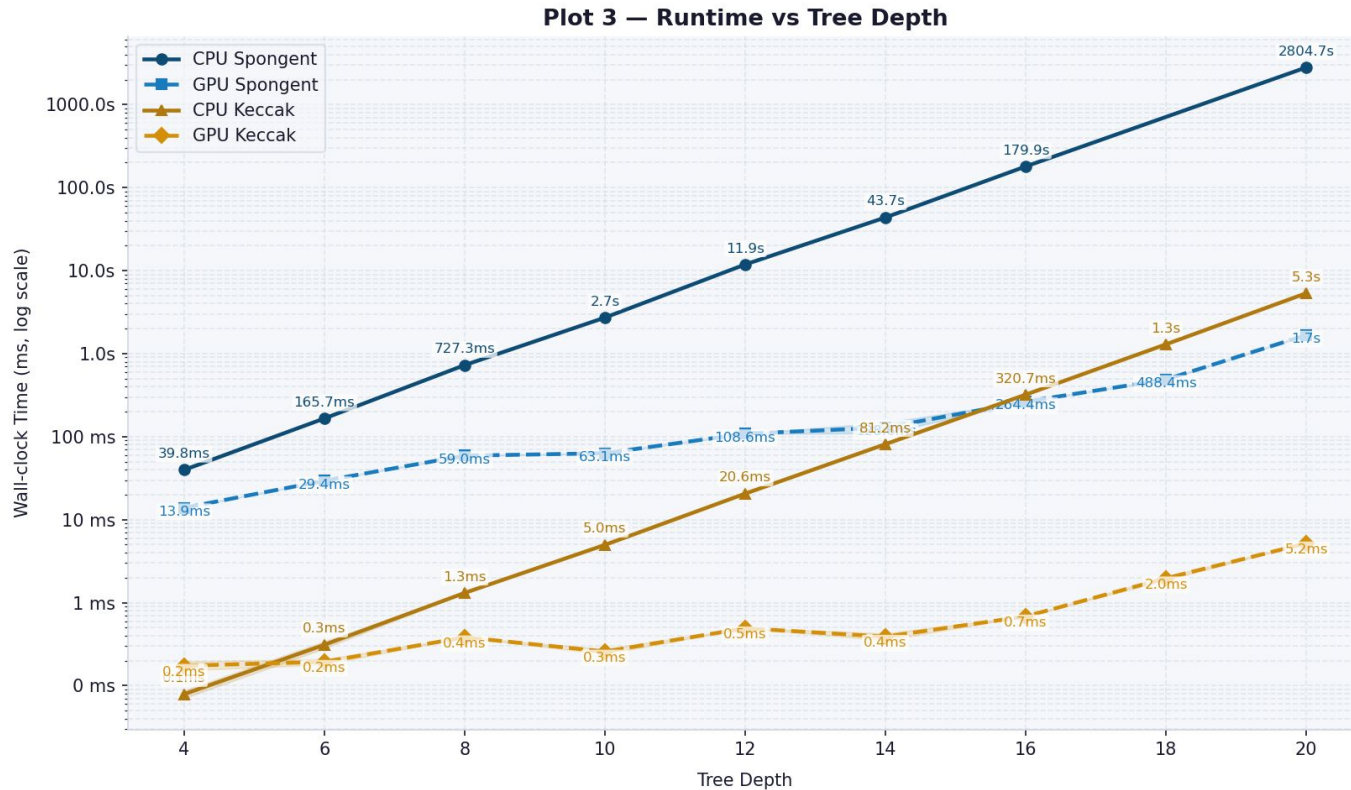
Benchmark Results



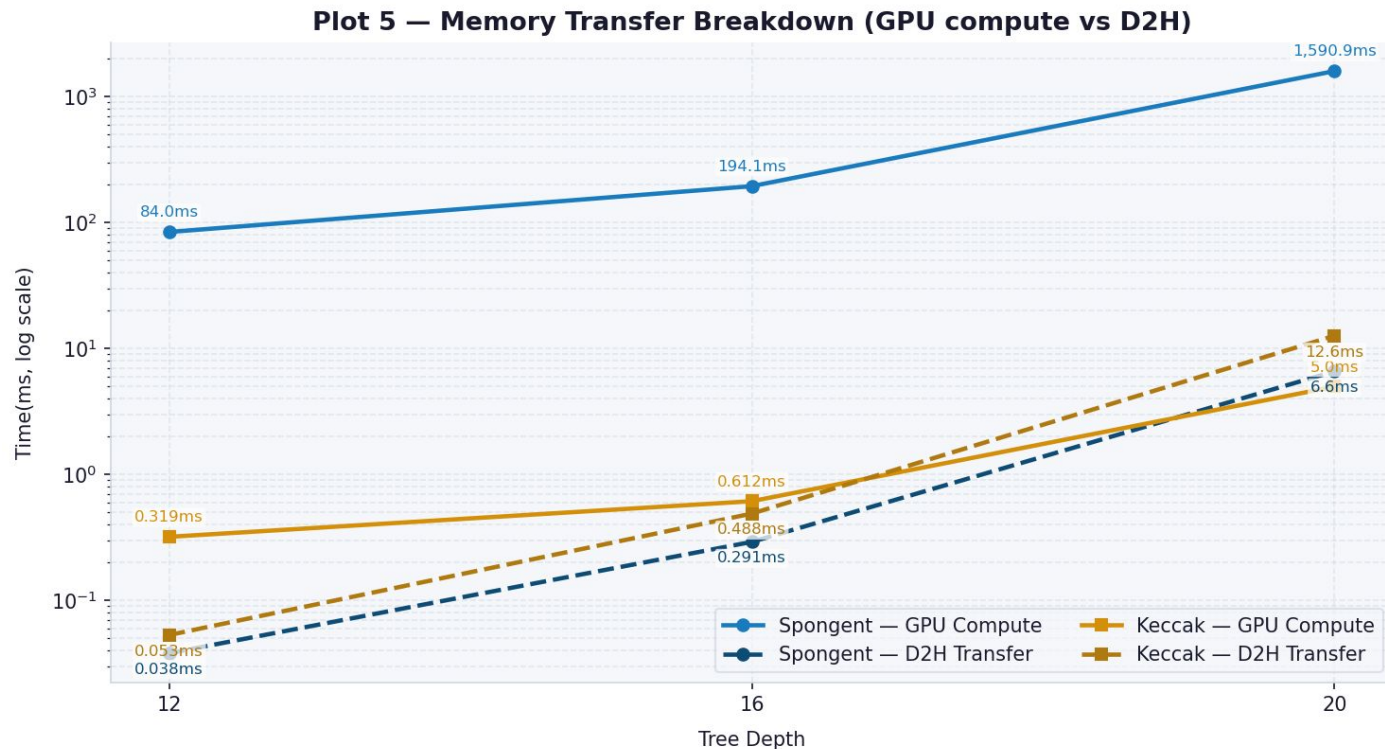
Plot 2 — Throughput vs Tree Depth



Benchmark Results



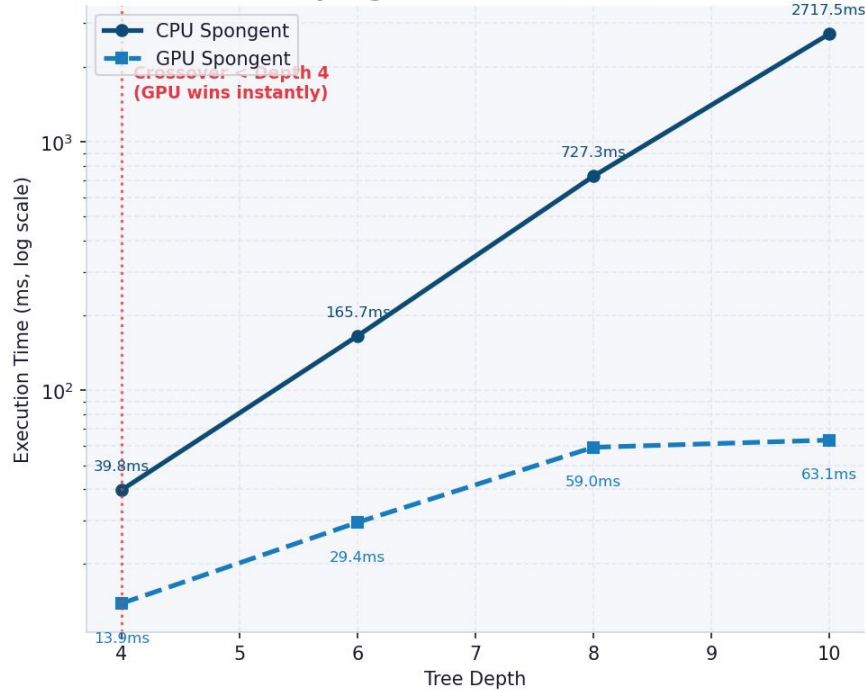
Benchmark Results



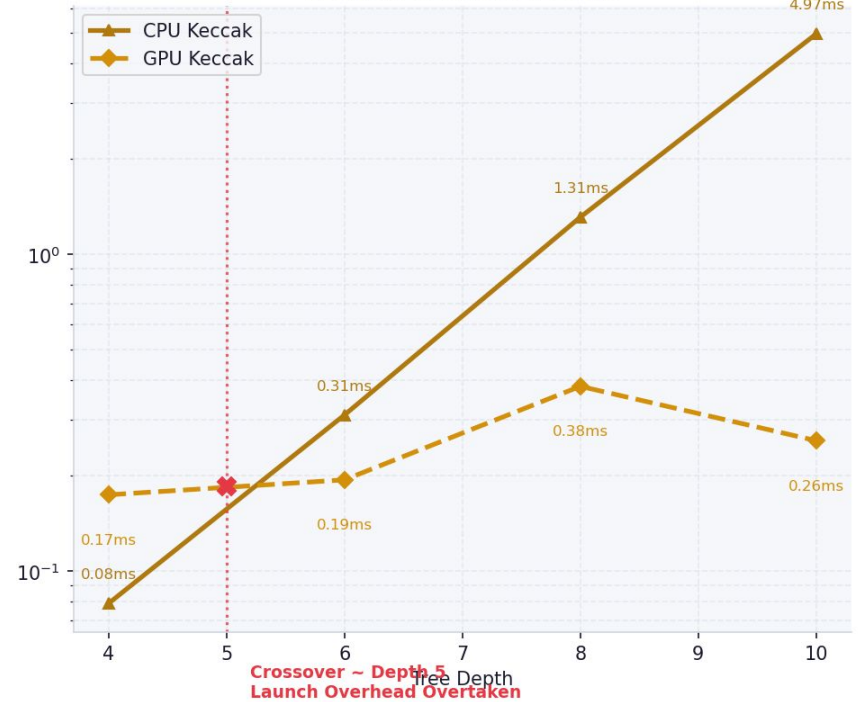


Crossover Analysis

Spongnet Crossover Zone



Keccak Crossover Zone



Performance Analysis: The Structural Ceiling



KECCAK: ALU-NATIVE EFFICIENCY

- **1:1 Mapping:** Steps are direct representations of CPU/GPU ALU operations, maximizing native execution.
- **Hardware vs Software:** Algorithmic requirements align with platform capabilities, ensuring no simulation overhead.
- **Native Advantage:** Throughput is limited only by hardware clock speed and available parallel cores.

SPONGENT: SIMULATION BOTTLENECK

- **Permutation Layer (pLayer):** In hardware design, this is an arrangement of physical wires with zero clock latency.
- **Software Penalty:** GPU execution requires ~136 instructions (shifts, masks, merging) to simulate one "free" wire layer.
- **Latency Hiding at Scale:** At higher depths (Depth 16+), massive GPU thread occupancy partially masks this penalty, narrowing the raw throughput gap to ~386x.

CRITICAL INSIGHT: This performance gap represents a **STRUCTURAL CEILING**. Because the disparity arises from the physical difference between wire routing and ALU operations, it is **NOT FIXABLE BY SOFTWARE OPTIMIZATION ALONE**.



Performance-Security Tradeoff





Validation & Correctness

- **Spongnet KAT**: official CHES 2011 test vector verified. pLayer bijectivity and LFSR sequence also independently confirmed.
- **Keccak KAT**: NIST FIPS 202 SHA3-256 empty string verified. GPU vs CPU equivalence tested across 10,000 random state vectors.
- **CPU↔GPU cross-validation**: 3 fixed seeds (all-zeros, all-ones, incrementing) at depths 0, 1, 4, 8, byte-for-byte match across all 511 nodes per tree (1,533 total nodes verified per PRF).
- **Domain separation**: left child $\neq$ right child confirmed for all seeds and depths. Output $\neq$ input seed confirmed.
- **GGM framework**: 18/18 structural tests with Dummy PRF: out-of-bounds access, null inputs, and tree layout verified independently of the hash function.
- **Test suite**: 90 named assertions across 4 test binaries; all pass



Conclusion

- **Scaling Works** : GPUs deliver exponential speedup as tree depth increases, provided the primitive allows for high-throughput execution.
- **Primitive Match** : **Keccak** is the clear choice for software/GPU acceleration and security wise too, as it is more fit for the hardware platform.
- **Optimization Limits** : While our 4 GPU optimizations (shared memory, coalescing) helped, the structural ceiling of Spongent remains immutable.
- **Performance-Security Trade-off**: Forcing a silicon-centric primitive into software causes thread divergence and opens the door to cache-timing side-channel leaks.



Thank you for listening!